

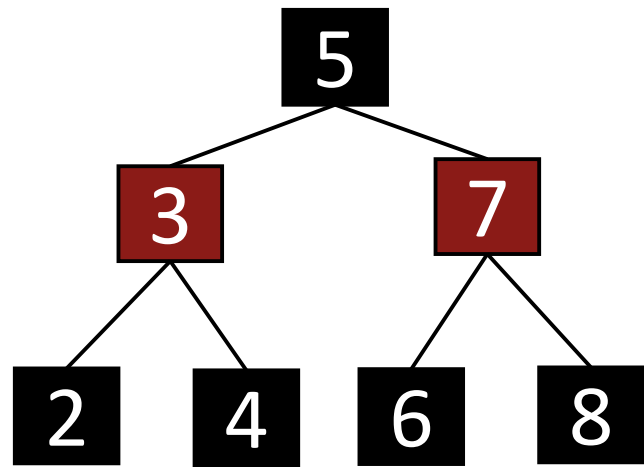


Red-Black Trees

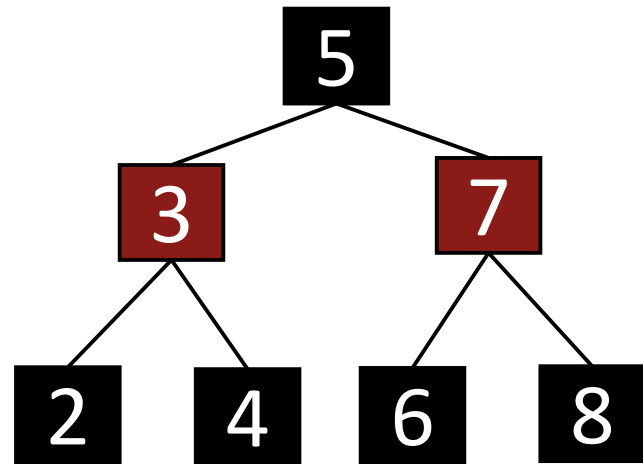
- AVL trees requires more rotations during insertion/deletion due to relatively strict balancing.
- What if we relax the constraint a bit and use some **proxy** of balancing?

Red-Black tree!

Maintain balance by stipulating that **black nodes** are balanced, and that there *aren't too many red nodes*.



- Every node is colored **red** or **black**.
- The root node is a **black** node.
- NIL children count as **black** nodes.
- Children of a **red** node are **black** nodes.
- For all nodes x :
 - all paths from x to NIL's have the same number of **black** nodes on them.

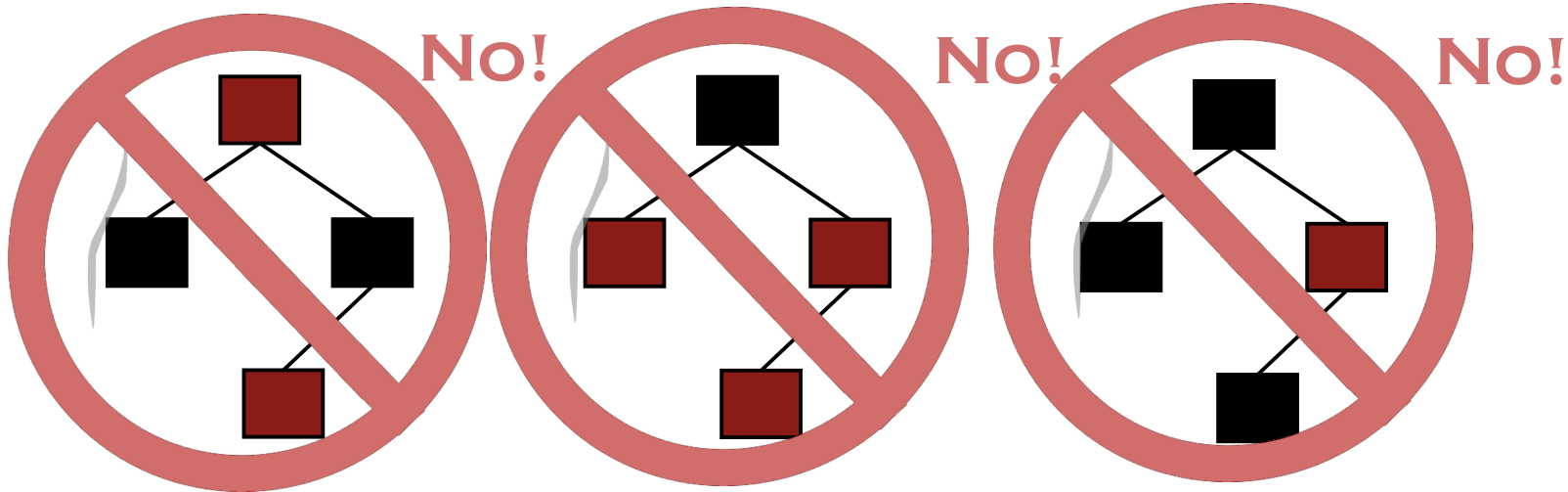


- Every node is colored **red** or **black**.
- The root node is a **black** node.
- NIL children count as **black** nodes.
- Children of a **red** node are **black** nodes.
- For all nodes x:
 - all paths from x to NIL's have the same number of **black** nodes on them.

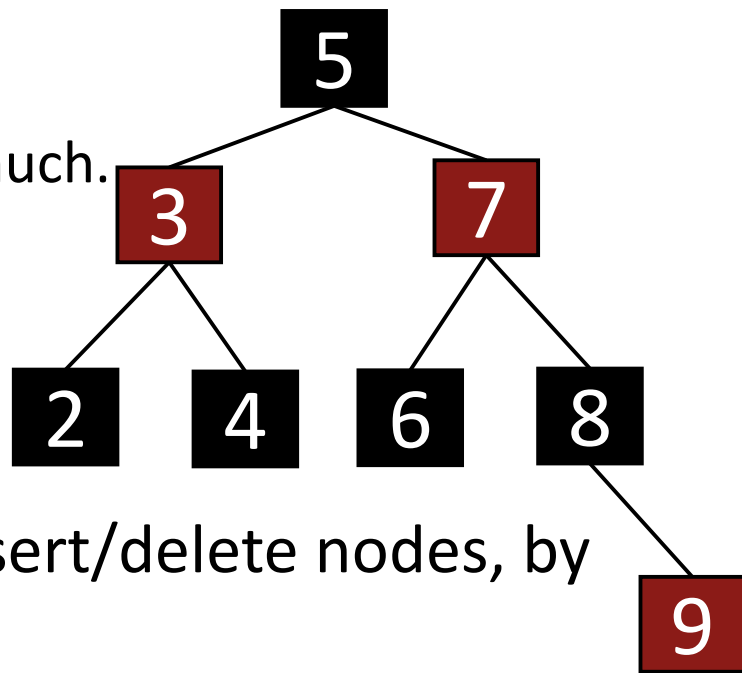
Which of these
are red-black trees?
(NIL nodes not drawn)

1 minute think

1 minute share

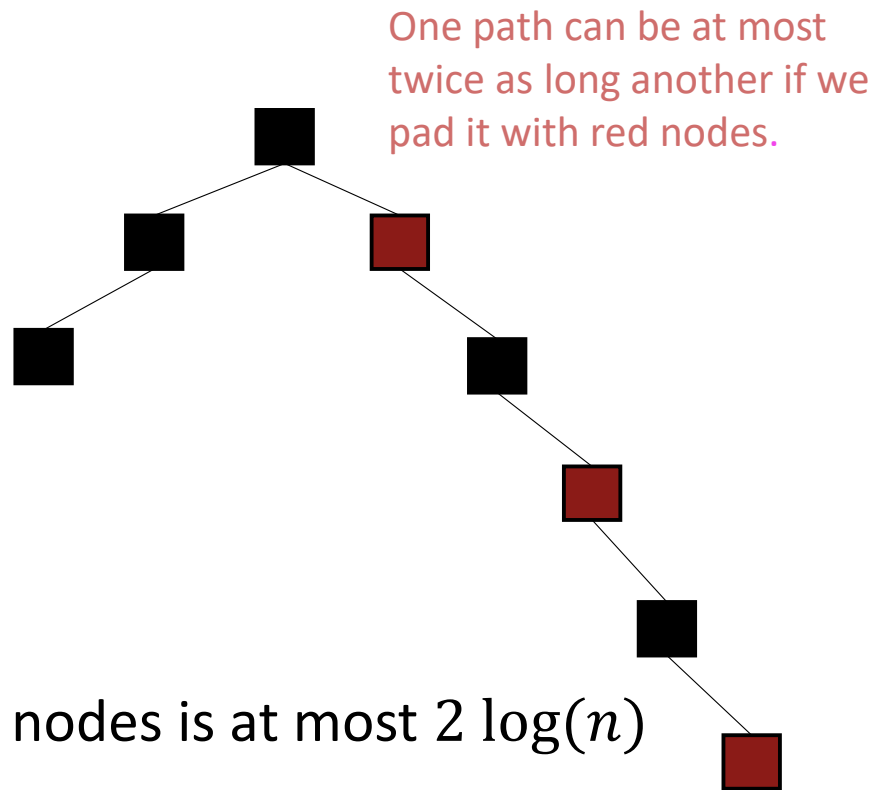


- This is pretty balanced.
 - The **black nodes** are balanced
 - The **red nodes** don't mess things up too much.



- We can maintain this property as we insert/delete nodes, by using rotations or color flipping.

- This is “pretty balanced”.



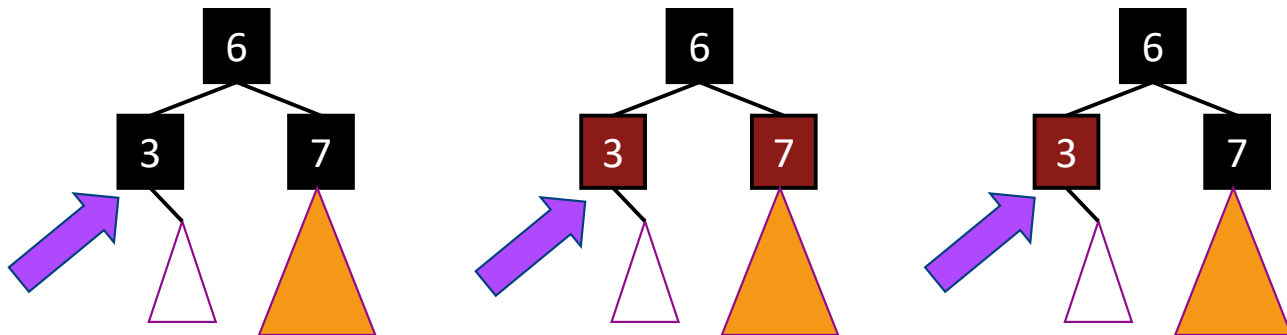
- Conjecture:
 - the height of a **red-black** tree with n nodes is at most $2 \log(n)$

The height of a RB-tree with n non-NIL nodes is at most $2\log_2(n + 1)$.

- Prove it?

- Since the insertion and deletion in RB Trees are complicated, you don't need to master the details of them.
 - You should know what the “proxy for balance” property is and why it ensures approximate balance.
 - You should know **that** this property can be efficiently maintained, but you do not need to know the details of how.

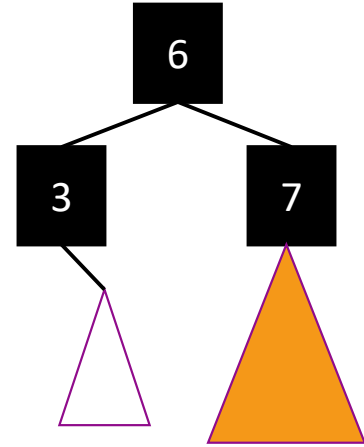
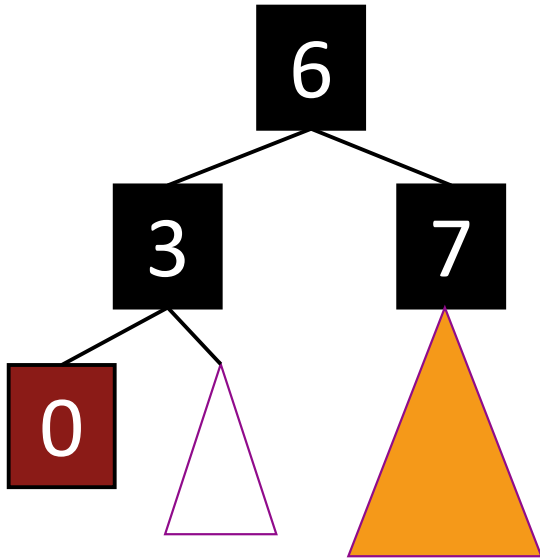
Many cases



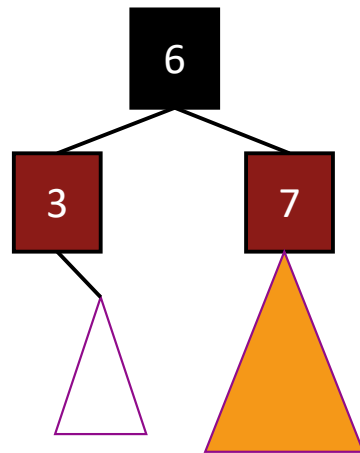
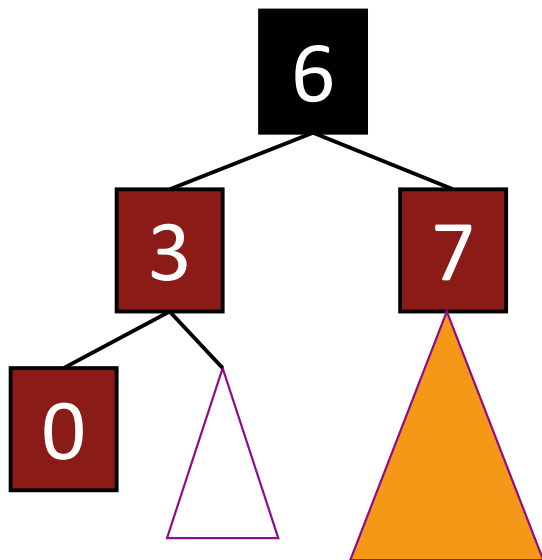
- Suppose we want to insert 0
- 3 “important” cases for different colorings of the existing tree, and there are 9 more cases for all of the various symmetries of these 3 cases.

Insert: Case 1

- Make a new **red node**.
- Insert it as you would normally.



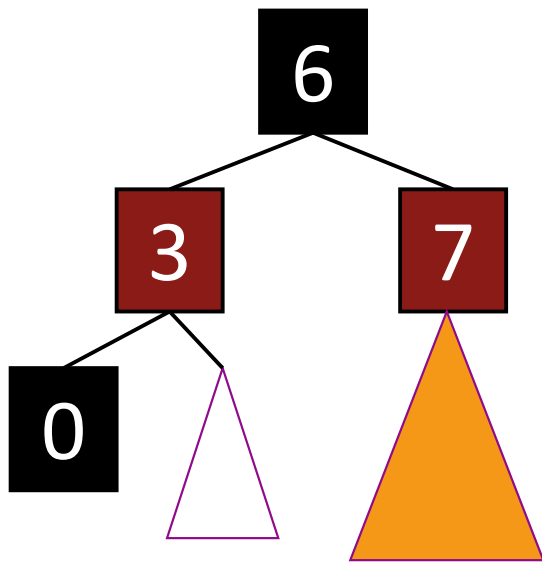
- Make a new **red node**.
- Insert it as you would normally?
- Fix things up if needed.



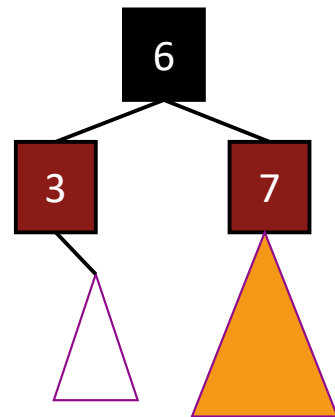
What if it looks like this?

Insert: Case 2

- Make a new **red node**.
- Insert it as you would normally?
- Fix things up if needed.



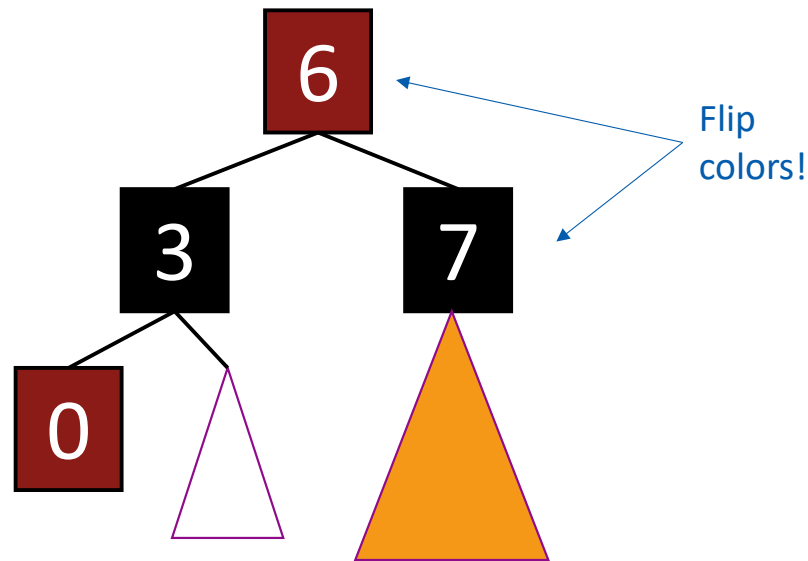
Can't we just insert
0 as a **black node**?



What if it looks like this?

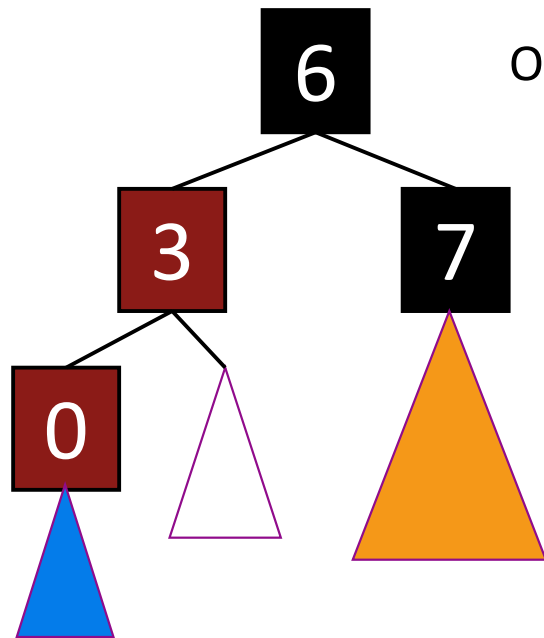
**One more black
node in this path!**

- An important observation: The root can be switched from red to black without violating any rule.



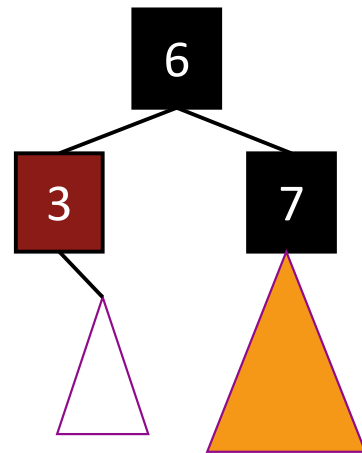
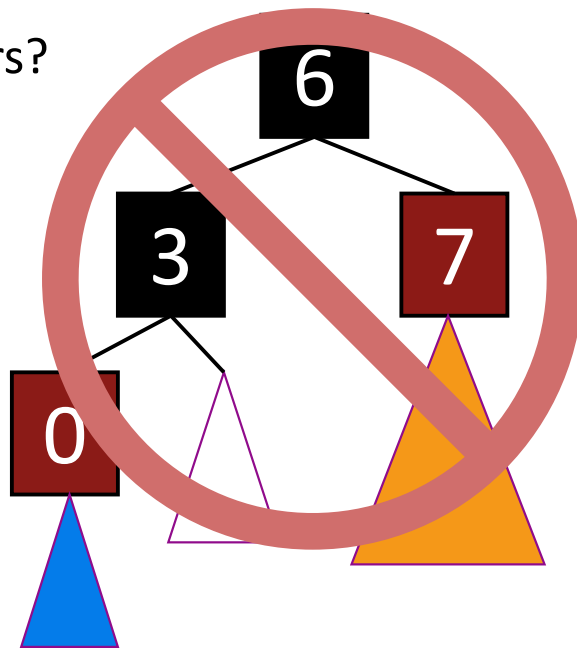
- Add 0 as a **red** node.
- Flip the colors of its parent and uncle.
- Pass the **red** to the top.
- Flip the color of the root from **red** to **black**.

- Make a new **red node**.
- Insert it as you would normally?
- Fix things up if needed.



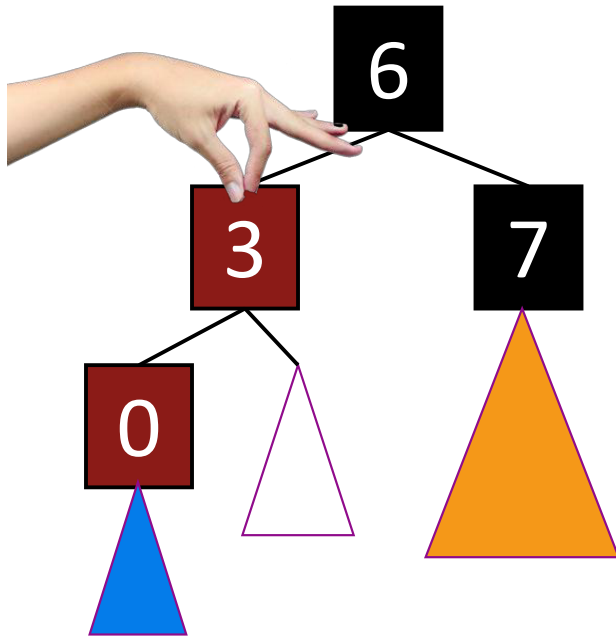
Only flip colors?

No!

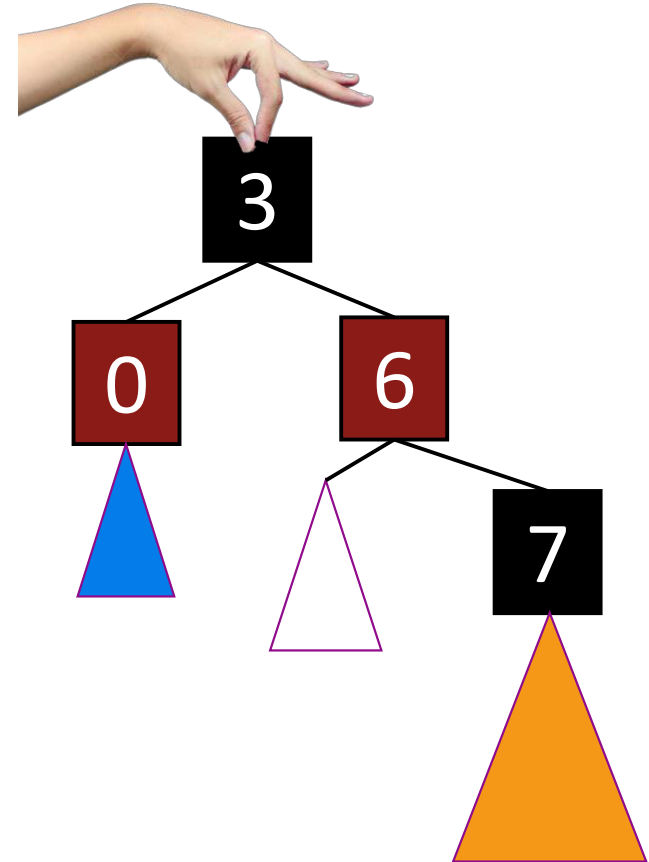


What if it looks like this?

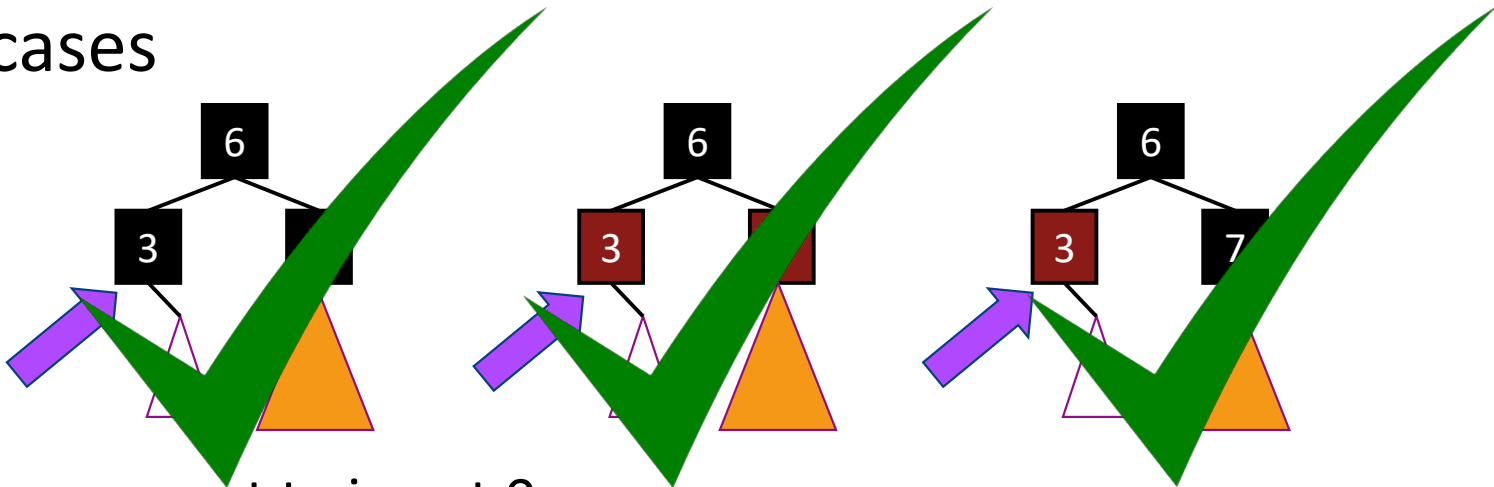
- Recall Rotations:



Rotate
+
Flip color



Many cases



- Suppose we want to insert 0
- 3 “important” cases for different colorings of the existing tree, and there are 9 more cases for all of the various symmetries of these 3 cases.